

Stationary Lag-1 Poisson GLMs: Dale Simulation, LASSO Fit, and FDR Edge Selection (Ver6)

Jan Balewski, NERSC

March 22, 2026

Abstract

This note documents the `stationaryLag1.ver6.lasso` pipeline: (1) `sim_daleLag1Poisson.py` generates Dale-law spike trains from a stationary lag-1 Poisson GLM; (2) `fit_lassoPoisson.py` fits a fully connected Poisson GLM with L_1 regularization on off-diagonal connectivity; (3) `selectEdges_FDR.py` combines bootstrap LASSO estimates with time-desynchronized (null) fits and applies row-wise Benjamini–Hochberg FDR to produce a sparse inferred matrix $\hat{A}$.

Contents

1 Synthetic spikes: <code>sim_daleLag1Poisson.py</code>	2
1.1 Connectivity and Dale structure	2
1.2 Stabilization (continuous Lyapunov heuristic)	2
1.3 Lag-1 Poisson GLM simulation	2
1.4 Frequency sorting and outputs	2
2 LASSO Poisson fitting: <code>fit_lassoPoisson.py</code>	2
2.1 Input and design matrix	2
2.2 Model and loss	3
2.3 Output file	3
3 FDR edge selection: <code>selectEdges_FDR.py</code>	3
3.1 Bootstrap inputs	3
3.2 Test statistic and null	3
3.3 Row-wise Benjamini–Hochberg and masked estimate	4
3.4 Output	4
4 Default hyperparameters (reference)	4

1 Synthetic spikes: `sim_daleLag1Poisson.py`

1.1 Connectivity and Dale structure

For N neurons with N_{exc} excitatory and $N_{\text{inh}} = N - N_{\text{exc}}$ inhibitory, target out-degrees ρ_j are drawn uniformly from an interval set by `--edge_prob` (fractions of N). An initial weight matrix is built in `gen_init_W`: for each presynaptic neuron j , ρ_j distinct postsynaptic targets are chosen at random; excitatory rows receive positive weights drawn uniformly in a band, inhibitory rows receive negative weights scaled by a fixed ratio g (code default $g = 2$). The diagonal is set to a negative value (default -1) to model self-inhibition.

1.2 Stabilization (continuous Lyapunov heuristic)

If the maximum real part of the eigenvalue spectrum of A is non-negative, `stabilize` iteratively adjusts *inhibitory* entries using gradients derived from continuous Lyapunov equations (see SI of the referenced neuroscience paper in-code) until $\max_i \text{Re } \lambda_i(A) < -\delta$ for a margin δ (code uses `eigenGap=0.1`). Small inhibitory weights are zeroed to avoid numerical issues.

1.3 Lag-1 Poisson GLM simulation

Spike counts $Y_t \in \mathbb{Z}_{\geq 0}^N$ are generated for $t = 0, \dots, T-1$ with bin width Δt (`--step_size`). Let A be the stabilized Dale matrix and B the bias (log-rate scale). The code uses

$$\eta_t = AY_{t-1} + B, \quad \tilde{\eta}_t = \text{clip}(\eta_t, -5, 5), \quad \mu_t = \exp(\tilde{\eta}_t) \Delta t, \quad Y_t \sim \text{Poisson}(\mu_t), \quad (1)$$

with $Y_0 \sim \text{Poisson}(\exp(B) \Delta t)$. Here $(AY_{t-1})_i = \sum_j A_{ij} Y_{t-1,j}$ matches the implementation `eta = A @ Y[t-1] + B`.

Biases B are drawn either uniformly in log of an idle rate range (`--idleRate`, default mode) or from a more structured frequency generator (`--expRate`).

1.4 Frequency sorting and outputs

Firing-rate statistics are computed with `UtilDalePoisson.estimate_rates`. Neurons are then *reordered by decreasing estimated firing rate*. All saved tensors use this **frequency-sorted** order: the permutation is recorded as `neur_freqIdx` (natural $\rightarrow$ sorted index) and `neur_revFreqIdx` (sorted $\rightarrow$ natural). The ground-truth matrix is $A' = PAP^\top$ and $B' = PB$ for the same permutation P .

`<dataName>.simTruth.npz (arrays). A_true, B_true, neur_freqIdx, neur_revFreqIdx.`

`<dataName>.spikes.npz (arrays). spikes (uint8,  $T \times N$ , frequency-sorted columns), single_rates, single_rates_var, single_rates_snr (note spelling), single_fano_fact.`

Metadata. `dale_conf` (network parameters), `evol_conf` (steps, Δt , rate options), `short_name`, `dale_simu_stats`; spike file includes `time_step_sec`, `data_type=simDale`, `var_time_window_sec`.

2 LASSO Poisson fitting: `fit_lassoPoisson.py`

2.1 Input and design matrix

The script reads `<inpPath>/<dataName>.spikes.npz`. Let $Y \in \mathbb{R}^{T \times N}$ be the spike matrix (already frequency-sorted). Training pairs are consecutive bins:

$$(x^{(t)}, y^{(t)}) = (Y_t, Y_{t+1}), \quad t = 0, \dots, T-2, \quad (2)$$

optionally truncated to `--num_samples` pairs. Optional `--dropDataFrac` randomly subsamples pairs.

Time desynchronization (null data for later FDR). If `--desyncTime` is used when fitting *separate* runs, each neuron’s time series is circularly shifted by an independent random amount, destroying cross-neuron temporal alignment while preserving marginal statistics. Those fits are saved with distinct filenames for use as nulls in Section 3.

2.2 Model and loss

`PoissonGLModel` parameterizes full $A \in \mathbb{R}^{N \times N}$ and $B \in \mathbb{R}^N$. For a batch of previous spikes Y_{prev} (rows = samples), predicted rates are

$$\mu = \exp(B^\top + Y_{\text{prev}} A^\top) \Delta t, \quad (3)$$

with the linear term clamped to $[-10, 10]$ before the exponential (implementation uses `addmm` with $A^\top$).

The training loss is a *rate-weighted* Poisson negative log-likelihood (see `poisson_nll_loss`): weights are inversely proportional to per-neuron estimated firing rates (clipped to $[0.1, 40]$ Hz from `single_rates`), then normalized to mean 1. This down-weights very active neurons relative to a uniform NLL.

With scalar `--L1_alpha` = α_{L1} , the total objective adds mean absolute value of *off-diagonal* entries of A :

$$\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha_{L1} \cdot \frac{1}{N(N-1)} \sum_{i \neq j} |A_{ij}|. \quad (4)$$

Optimization uses Adam on all entries of A and B , with optional linear learning-rate decay (`train.Poisson_model`). Multi-GPU training uses PyTorch `DistributedDataParallel`.

2.3 Output file

Rank 0 writes `<outPath>/<fitName>.lassoFit.npz` (default `fitName` is `<dataName>--<randomhash>`). Arrays include `A_lasso`, `B_lasso`, loss curves, and learning rates. Metadata embeds the spike file name/path, hyperparameters (`batch_size`, `num_epochs`, `L1_alpha`, `step_size`, etc.) and augments the spike metadata with `fit_type=lasso`.

3 FDR edge selection: selectEdges_FDR.py

3.1 Bootstrap inputs

The script expects a set of LASSO fits produced offline (e.g. bootstrap resamples of the spike train or repeated random seeds). Filenames follow:

- Real (aligned-time) bootstraps: `<dataName>-boots{k}.lassoFit.npz` for $k = 0, \dots, K_{\text{real}} - 1$.
- Null / desynchronized-time bootstraps: `<dataName>-desync{k}.lassoFit.npz` for $k = 0, \dots, K_{\text{desync}} - 1$.

The counts K_{real} and K_{desync} are set by `--num_bootstraps` (one integer duplicated for both, or two integers).

3.2 Test statistic and null

For each edge (i, j) , $i \neq j$, the observed statistic is the median of $|A_{ij}^{(k)}|$ across real bootstraps k . For the null, all magnitudes $|\tilde{A}_{i'j}^{(k)}|$ from desync fits for fixed postsynaptic row i are pooled (all columns j , all k). An empirical p -value is

$$p_{ij} = \frac{\#\{\text{null values} \geq \text{observed median}\}}{\#\text{null values}}. \quad (5)$$

3.3 Row-wise Benjamini–Hochberg and masked estimate

For each row i , Benjamini–Hochberg FDR at level α (`--alphaFDR`, default 0.002) is applied to $\{p_{ij}\}_{j \neq i}$. This yields a binary mask M_{ij} on off-diagonals. Diagonal entries are always kept in the mask for downstream averaging.

Across real bootstraps, the script computes elementwise means $\bar{A}$, $\bar{B}$ and standard deviations. The **reported** connectivity is $\bar{A}$ with non-selected off-diagonal entries zeroed:

$$\hat{A}_{ij} = \bar{A}_{ij} M_{ij}, \quad i \neq j, \quad (6)$$

with diagonal unmasked.

3.4 Output

`<inpPath>/<dataName>.FDRselected.npz` contains `E_mask`, `A_avr`, `A_std`, `B_avr`, `B_std`, `E_pval`, `summary`, and loss arrays copied from the first bootstrap for convenience. Metadata records `edge_selector={FDR,  $\alpha$ }`.

4 Default hyperparameters (reference)

Script	Parameter	Typical default
<code>sim_daleLag1Poisson</code>	<code>--step_size</code>	0.01 s
	<code>--spectralR</code>	1.0
	<code>--edge_prob</code>	[0.05, 0.2]
<code>fit_lassoPoisson</code>	<code>--lr</code>	10^{-3}
	<code>--L1_alpha</code>	10^{-3}
	<code>--batch_size</code>	2048
	<code>--num_epochs</code>	7
<code>selectEdges_FDR</code>	<code>--alphaFDR</code>	0.002

Table 1: Illustrative defaults; see each script’s `argparse` for authoritative values.